

Prompt: Módulo de Agencias de Viaje

Contexto

Maran Suites necesita gestionar agencias de viaje similares a empresas, pero con lógica específica:

- Un huésped puede tener empresa Y agencia asociadas simultáneamente
 - La reserva vincula ambas: empresa para consumos extras, agencia para el alojamiento
 - Las agencias cobran comisión porcentual sobre el alojamiento
 - Las agencias tienen cuenta corriente (crédito acumulado) igual que empresas
 - Reporte de volumen de reservas por agencia
-

PASO 1 — Schema: nueva tabla `agencies` en `shared/schema.ts`

Agregar después de la tabla `companies` (buscar `export const companies = pgTable`):

```
// Travel Agencies
export const agencies = pgTable("agencies", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  razonSocial: text("razon_social").notNull(),
  nombreFantasia: text("nombre_fantasia"),
  direccion: text("direccion"),
  pais: text("pais").default("Argentina"),
  codigoPostal: text("codigo_postal"),
  localidad: text("localidad"),
  provincia: text("provincia"),
  telefono: text("telefono"),
  email: text("email"),
  cuilCuit: text("cuil_cuit").notNull(),
  numeroFiscal: text("numero_fiscal"),
  condicionIva: text("condicion_iva").$type<IvaCondition>().default("responsable_"),
  contactName: text("contact_name"),
  contactEmail: text("contact_email"),
  contactPhone: text("contact_phone"),
});
// Específico de agencias
```

```

    commissionRate: decimal("commission_rate", { precision: 5, scale: 2 }).default(
    creditLimit: decimal("credit_limit", { precision: 12, scale: 2 }).default("0"),
    paymentTermDays: integer("payment_term_days").default(30),
    notes: text("notes"),
    isActive: text("is_active").default("true"),
    createdAt: timestamp("created_at"),
  });

export const insertAgencySchema = createInsertSchema(agencySchema).omit({ id: true })
export type InsertAgency = z.infer<typeof insertAgencySchema>;
export type Agency = typeof agencySchema.$inferSelect;

```

PASO 2 — Agregar `agencyId` a tablas existentes en

`shared/schema.ts`

En tabla `reservations` (buscar `companyId: varchar("company_id")` en `reservations`):

```

companyId: varchar("company_id"),
agencyId: varchar("agency_id"),    // ← agregar después de companyId

```

En tabla `guests` (buscar `companyId: varchar("company_id")` en `guests`):

```

companyId: varchar("company_id"),
agencyId: varchar("agency_id"),    // ← agregar después de companyId

```

PASO 3 — Migración de base de datos

Agregar en el archivo de migración (o ejecutar directamente):

```

CREATE TABLE IF NOT EXISTS agencies (
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
  razon_social TEXT NOT NULL,
  nombre_fantasia TEXT,
  direccion TEXT,
  pais TEXT DEFAULT 'Argentina',
  codigo_postal TEXT,
  localidad TEXT,

```

```

    provincia TEXT,
    telefono TEXT,
    email TEXT,
    cuil_cuit TEXT NOT NULL,
    numero_fiscal TEXT,
    condicion_iva TEXT DEFAULT 'responsable_inscripto',
    contact_name TEXT,
    contact_email TEXT,
    contact_phone TEXT,
    commission_rate DECIMAL(5,2) DEFAULT 0,
    credit_limit DECIMAL(12,2) DEFAULT 0,
    payment_term_days INTEGER DEFAULT 30,
    notes TEXT,
    is_active TEXT DEFAULT 'true',
    created_at TIMESTAMP DEFAULT NOW()
);

ALTER TABLE reservations ADD COLUMN IF NOT EXISTS agency_id VARCHAR;
ALTER TABLE guests ADD COLUMN IF NOT EXISTS agency_id VARCHAR;

```

PASO 4 — Storage: agregar métodos en `server/storage.ts`

Interfaz (buscar `updateCompany` en la interfaz `IStorage` y agregar debajo):

```

getAgencies(): Promise<Agency[]>;
getAgency(id: string): Promise<Agency | undefined>;
createAgency(agency: InsertAgency): Promise<Agency>;
updateAgency(id: string, agency: Partial<InsertAgency>): Promise<Agency | undefined>;
deleteAgency(id: string): Promise<boolean>;
getAgencyStats(agencyId: string): Promise<{ totalReservations: number; totalRevenue: number }>;

```

Implementación (agregar después de los métodos de companies):

```

async getAgencies(): Promise<Agency[]> {
    return await db.select().from(agency).orderBy(agency.razonSocial);
}

async getAgency(id: string): Promise<Agency | undefined> {
    const [agency] = await db.select().from(agency).where(eq(agency.id, id));
    return agency;
}

```

```

async createAgency(agency: InsertAgency): Promise<Agency> {
  const [created] = await db.insert(agency).values({
    ...agency,
    createdAt: new Date(),
  }).returning();
  return created;
}

async updateAgency(id: string, agency: Partial<InsertAgency>): Promise<Agency | u
  const safeData: Record<string, any> = {};
  const protectedFields = ["id", "createdAt"];
  for (const [key, value] of Object.entries(agency)) {
    if (protectedFields.includes(key) || value === undefined) continue;
    safeData[key] = value;
  }
  if (Object.keys(safeData).length === 0) return undefined;
  const [updated] = await db.update(agency).set(safeData).where(eq(agency.id,
  return updated;
}

async deleteAgency(id: string): Promise<boolean> {
  const result = await db.delete(agency).where(eq(agency.id, id));
  return (result.rowCount ?? 0) > 0;
}

async getAgencyStats(agencyId: string): Promise<{ totalReservations: number; tota
  const agencyReservations = await db.select().from(reservations)
    .where(and(
      eq(reservations.agencyId, agencyId),
      ne(reservations.status, "cancelled")
    ));

  const agency = await this.getAgency(agencyId);
  const commissionRate = parseFloat(agency?.commissionRate ?? "0") / 100;

  let totalRevenue = 0;
  for (const r of agencyReservations) {
    const nights = r.nights || 1;
    const rate = parseFloat(r.baseRatePerNight ?? "0");
    totalRevenue += nights * rate;
  }

  return {
    totalReservations: agencyReservations.length,
    totalRevenue: totalRevenue.toFixed(2),
    pendingCommission: (totalRevenue * commissionRate).toFixed(2),
  };
}

```

```
}
```

PASO 5 — API Routes en `server/routes.ts`

Agregar después de las rutas de `/api/companies`:

```
// — AGENCIES —————
app.get("/api/agencies", async (req, res) => {
  try {
    const agenciesList = await storage.getAgencies();
    res.json(agenciesList);
  } catch (error) {
    res.status(500).json({ error: "Error fetching agencies" });
  }
});

app.get("/api/agencies/:id", async (req, res) => {
  try {
    const agency = await storage.getAgency(req.params.id);
    if (!agency) return res.status(404).json({ error: "Agency not found" });
    res.json(agency);
  } catch (error) {
    res.status(500).json({ error: "Error fetching agency" });
  }
});

app.get("/api/agencies/:id/stats", async (req, res) => {
  try {
    const stats = await storage.getAgencyStats(req.params.id);
    res.json(stats);
  } catch (error) {
    res.status(500).json({ error: "Error fetching agency stats" });
  }
});

app.post("/api/agencies", async (req, res) => {
  try {
    const agency = await storage.createAgency(req.body);
    res.status(201).json(agency);
  } catch (error) {
    res.status(500).json({ error: "Error creating agency" });
  }
});
```

```

app.patch("/api/agencies/:id", async (req, res) => {
  try {
    const existing = await storage.getAgency(req.params.id);
    if (!existing) return res.status(404).json({ error: "Agency not found" });
    const { id, createdAt, ...updateData } = req.body;
    const agency = await storage.updateAgency(req.params.id, updateData);
    res.json(agency);
  } catch (error) {
    res.status(500).json({ error: "Error updating agency" });
  }
});

app.delete("/api/agencies/:id", async (req, res) => {
  try {
    const deleted = await storage.deleteAgency(req.params.id);
    if (!deleted) return res.status(404).json({ error: "Agency not found" });
    res.json({ success: true });
  } catch (error) {
    res.status(500).json({ error: "Error deleting agency" });
  }
});

```

PASO 6 — Frontend: página de Agencias

Crear `client/src/pages/agencies.tsx`. Esta página es idéntica a `companies.tsx` con estas diferencias:

1. **Campo extra:** `commissionRate` (porcentaje, number input 0-100)
2. **Sin campo:** `inscripcionNacional`, `inscripcionProvincial` (no aplica para agencias)
3. **Columna extra en la lista:** "Comisión %"
4. **Stats por agencia:** Total reservas | Ingresos totales | Comisión pendiente
5. **Título:** "Agencias de Viaje" en lugar de "Empresas"

```

// Esquema del form de agencia
const agencyFormSchema = insertAgencySchema.extend({
  razonSocial: z.string().min(1, "Razón social requerida"),
  cuilCuit: z.string().min(1, "CUIT requerido"),
  commissionRate: z.coerce.number().min(0).max(100).default(0),
});
type AgencyFormData = z.infer<typeof agencyFormSchema>;

```

Estructura de la página:

- Header: "Agencias de Viaje" + botón "Nueva Agencia"
 - Buscador por nombre/CUIT
 - Tabla con columnas: Razón Social | Contacto | Teléfono | Comisión % | Reservas | Estado | Acciones
 - Al hacer clic en una agencia → panel lateral o modal con:
 - Datos de la agencia
 - Stats: total reservas, ingresos generados, comisión acumulada
 - Lista de reservas asociadas (con fecha, huésped, habitación, monto, comisión calculada)
 - Form de crear/editar igual que companies pero con campo `commissionRate`
-

PASO 7 — Componente `AgencySelector`

Crear similar a `CompanySelector`. Buscar `CompanySelector` en el código y replicar con estos cambios:

```
// AgencySelector.tsx — igual que CompanySelector pero para agencias
// Muestra: nombre + porcentaje de comisión entre paréntesis
// Ej: "Punto Turístico (10%)"

function AgencySelector({ selectedAgency, onSelect, onCreateNew, onClear }) {
  // ... misma estructura que CompanySelector
  // En el display mostrar:
  <span>{agency.nombreFantasia || agency.razonSocial}
    {agency.commissionRate && parseFloat(agency.commissionRate) > 0 && (
      <Badge variant="outline" className="ml-2 text-xs">
        {agency.commissionRate}% comisión
      </Badge>
    )}
  </span>
}
```

PASO 8 — Agregar `AgencySelector` al form de reserva

En `ReservationFormDialog` , buscar el bloque del `CompanySelector` :

```
<CompanySelector
  selectedCompany={selectedCompany}
  onSelect={(company) => { ... }}
  ...
/>
```

Agregar debajo el `AgencySelector` :

```
{/* Agencia — para facturación del alojamiento */}
<AgencySelector
  selectedAgency={selectedAgency}
  onSelect={(agency) => {
    setSelectedAgency(agency);
    setFormData((prev) => ({ ...prev, agencyId: agency.id }));
  }}
  onCreateNew={(agency) => createAgencyMutation.mutate(agency)}
  onClear={() => {
    setSelectedAgency(null);
    setFormData((prev) => ({ ...prev, agencyId: "" }));
  }}
/>
```

Agregar estado en el form:

```
const [selectedAgency, setSelectedAgency] = useState<Agency | null>(null);

// En el useEffect que inicializa el form al editar:
setSelectedAgency(reservation?.agency || null);

// En formData inicial:
agencyId: reservation?.agencyId || "",

// En la mutation de crear/editar, incluir agencyId:
agencyId: formData.agencyId || null,
```

PASO 9 — Agregar agencia a la ficha del huésped

En el formulario/modal de huésped (`GuestFormDialog` o similar), buscar donde está el selector de empresa del huésped y agregar debajo el selector de agencia:


```

{/* Empresa habitual del huésped */}
<CompanySelector
  selectedCompany={guestCompany}
  onSelect={(c) => setFormData(prev => ({ ...prev, companyId: c.id })))}
  onClear={() => setFormData(prev => ({ ...prev, companyId: "" })))}
/>

{/* Agencia habitual del huésped */}
<AgencySelector
  selectedAgency={guestAgency}
  onSelect={(a) => setFormData(prev => ({ ...prev, agencyId: a.id })))}
  onClear={() => setFormData(prev => ({ ...prev, agencyId: "" })))}
/>

```

PASO 10 — Mostrar agencia en ReservationDetailModal y ReservationDetailDialog

En el detalle de reserva, junto a donde se muestra la empresa, agregar la agencia:

```

{/* Empresa / Agencia */}
<div className="grid grid-cols-2 gap-4">
  {reservation.company && (
    <div className="flex items-center gap-2">
      <Building2 className="h-4 w-4 text-muted-foreground" />
      <div>
        <p className="text-xs text-muted-foreground">Empresa (consumos)</p>
        <p className="text-sm font-medium">
          {reservation.company.nombreFantasia || reservation.company.razonSocial}
        </p>
      </div>
    </div>
  )}
  {reservation.agency && (
    <div className="flex items-center gap-2">
      <Plane className="h-4 w-4 text-muted-foreground" />
      <div>
        <p className="text-xs text-muted-foreground">Agencia (alojamiento)</p>
        <p className="text-sm font-medium">
          {reservation.agency.nombreFantasia || reservation.agency.razonSocial}
          {reservation.agency.commissionRate && parseFloat(reservation.agency.commissionRate) > 0 ? (
            <span className="text-xs text-muted-foreground ml-1">
              ({reservation.agency.commissionRate}% comisión)
            </span>
          ) : null}
        </p>
      </div>
    </div>
  )}

```

```
    })
  </p>
</div>
</div>
})
</div>
```

PASO 11 — Enriquecer reserva con agencia en backend

En `enrichReservation` (buscar esa función en `storage.ts`), agregar la agencia junto a la empresa:

```
// Buscar donde se enriche company y agregar agency:
const agency = r.agencyId
  ? await db.select().from(agency).where(eq(agency.id, r.agencyId)).then(c =>
    : undefined;

// En el objeto retornado:
return {
  ...r,
  guest,
  room,
  roomType,
  company,
  agency, // ← agregar
  // ...
};
```

PASO 12 — Sidebar y rutas

En `App.tsx` (rutas):

```
import AgenciesPage from "@pages/agencies";
// ...
<Route path="/agencies" component={AgenciesPage} />
```

En el Sidebar, buscar el ítem de Empresas y agregar Agencias debajo:

```
<SidebarMenuItem>
```

```

<SidebarMenuButton asChild isActive={location === "/agencies"} data-testid="nav
  <Link href="/agencies">
    <Plane className="h-5 w-5" />
    <span>Agencias</span>
  </Link>
</SidebarMenuButton>
</SidebarMenuItem>

```

PASO 13 — Reporte de volumen por agencia

En la página de Reportes o en la propia página de Agencias, agregar una sección de reporte:

```

// Endpoint ya creado: GET /api/agencies/:id/stats
// Para reporte global, agregar en routes.ts:

app.get("/api/agencies/report", async (req, res) => {
  try {
    const { startDate, endDate } = req.query;
    const allAgencies = await storage.getAgencies();

    const report = await Promise.all(
      allAgencies.map(async (agency) => {
        const agencyReservations = await db.select().from(reservations)
          .where(and(
            eq(reservations.agencyId, agency.id),
            ne(reservations.status, "cancelled"),
            startDate ? gte(reservations.checkInDate, startDate as string) : unde
            endDate ? lte(reservations.checkInDate, endDate as string) : undefine
          ).filter(Boolean));

        const commissionRate = parseFloat(agency.commissionRate ?? "0") / 100;
        let revenue = 0;
        for (const r of agencyReservations) {
          revenue += (r.nights || 1) * parseFloat(r.baseRatePerNight ?? "0");
        }

        return {
          agencyId: agency.id,
          agencyName: agency.nombreFantasia || agency.razonSocial,
          commissionRate: agency.commissionRate,
          totalReservations: agencyReservations.length,
          totalRevenue: revenue.toFixed(2),
          totalCommission: (revenue * commissionRate).toFixed(2),

```

```

        };
    })
);

// Ordenar por mayor cantidad de reservas
report.sort((a, b) => b.totalReservations - a.totalReservations);
res.json(report);
} catch (error) {
    res.status(500).json({ error: "Error generating agency report" });
}
});

```

UI del reporte en la página de Agencias — tabla con filtro por rango de fechas:

Agencia	Comisión %	Reservas	Ingresos	Comisión a pagar
Punto Turístico	10%	45	\$180.000	\$18.000
Viajes Sol	8%	23	\$92.000	\$7.360

Resumen de archivos a crear/modificar

Archivo	Acción
shared/schema.ts	Agregar tabla <code>agencies</code> , <code>agencyId</code> en <code>reservations</code> y <code>guests</code>
server/storage.ts	Agregar métodos CRUD de <code>agencies</code> + <code>enrichReservation</code>
server/routes.ts	Agregar rutas <code>/api/agencies</code>
client/src/pages/agencies.tsx	Crear página nueva (basada en <code>companies.tsx</code>)
client/src/components/AgencySelector.tsx	Crear componente selector
client/src/components/ReservationFormDialog.tsx	Agregar <code>AgencySelector</code>
client/src/pages/guests.tsx	Agregar <code>agencyId</code> en ficha huésped

<code>client/src/App.tsx</code>	Agregar ruta + sidebar
Migración SQL	Ejecutar ALTER TABLE y CREATE TABLE